

Exploring LSTM based Autoencoders for Gravitational Waves Anomaly Detection

Luigi Belli, Giovanni Andrea Maida

Abstract—Gravitational Waves are faint signals from the cosmos carrying valuable and precise information about dramatic events in the Universe such as kilonovae: tremendous explosions due to the merging of extremely dense objects like Black Holes and Neutron Stars. However, the sensitivity required to measure such events is so high, that noise often eclipses the signal, despite the sophisticated countermeasures adopted by the interferometers. Traditional techniques to detect and study Gravitational Waves prove to be computationally inefficient, where unsupervised deep learning strategies might establish a new standard for the detection of these anomalies. In this work we study the application of Long Short-Term Memory Autoencoders to Time Series Anomaly Detection, and aim to improve on the existing strategies. We modify a standard Autoencoder design by testing skip connections, Time-Delay strategies, and introducing a new regularization term, all to improve the reconstruction power of this Neural Network. While we explored a few possibilities with decent results (76% of recall, and 64% of precision) on a synthetic dataset, we strongly believe that the answer to efficiently decode Gravitational Waves is yet to be found, and lies in these deep learning models. In fact, this work could theoretically be applied to any Time Series scenario.

Index Terms—Unsupervised Learning, Gravitational Waves, Long Short-Term Memory Autoencoder, Time Series Anomaly Detection.

I. INTRODUCTION

The joint operation of the LIGO, Virgo, and KAGRA detectors [1] has produced an unprecedented volume of data [2]. Since these detectors operate at extreme sensitivities to capture the faint intensity of the gravitational wave (GW) signals, the resulting datasets are inevitably dominated by noise. Traditional detection methods rely on matched filtering, which compares pre-computed signal templates with data, but as the template bank expands, it becomes less computationally efficient.

Unsupervised deep learning methods for Time Series Anomaly Detection (TSAD) aim to eliminate the bias associated with pre-defined anomaly templates, and can be trained efficiently using exclusively background noise. Given the nature of the GW time-series data, Long Short-Term Memory (LSTM) architectures are naturally suited for this domain. By capturing the temporal dependencies of the underlying background noise, LSTM-based autoencoders (AE) aim to minimize reconstruction error, offering a theoretical advantage over standard CNN AEs and supervised techniques, providing a faster and more efficient framework to identify GW anomalies.

This study serves as a direct extension of the work presented by Moreno et al. [3]. Their study demonstrates the feasibility of using unsupervised learning for GW detection, identifying

in LSTM AEs the most promising architecture. Motivated by these results, we present a comparative analysis of LSTM-based AE architectures to identify the optimal configuration for unsupervised GW detection. Starting from the best model from [3], the objective is to improve detection capabilities by introducing architectural enhancements, such as skip connections, multivariate inputs, and domain-specific regularization methods. The relevance of this work is about the evaluation of the efficacy of LSTM AEs for establishing a new standard in GW TSAD. This research offers an updated baseline for anomaly detection techniques in GW astronomy, offering insights on the practical application and its limitations.

To summarize, the main contribution of this paper is about benchmarking and extension: we compare our models against the foundational AE architecture proposed by [3], extending their results by exploring architectural variations.

This paper is structured as follows: Section II outlines related work, Section III describes the detectors and synthetic signals, and Section IV details the processing pipeline and model architectures. Section V presents the results, followed by concluding remarks in Section VI.

II. RELATED WORK

Although Matched Filtering (MF) remains the gold standard for gravitational wave (GW) detection, its reliance on template banks imposes severe computational limits as detector sensitivity increases, and signal characterizations become more complex. Furthermore, MF is highly susceptible to non-stationary instrumental artifacts, or glitches, mimicking astrophysical signals, and degrading overall data quality [4].

To overcome these limitations, early deep learning efforts utilize supervised Convolutional Neural Networks (CNNs) trained from scratch on the crowdsourced Gravity Spy dataset [5].

While these models struggle with small sample sizes, and require manual redesign for novel anomalies, the introduction of Transfer Learning via deep pre-trained models successfully reduces training times, requires fewer labeled examples, and significantly reduces the classification error rate [6].

However, supervised methods still fundamentally rely on predefined templates. Shifting the paradigm from targeted signal matching to unmodeled anomaly detection, unsupervised CNN Autoencoders (CNN-AE) are developed to identify GWs as unexpected anomalies within detector noise. This unsupervised approach proves capable of generalizing to unknown signals, and suggests the future application of recurrent neural networks for time-series data [7].

Building directly on this premise, Moreno et al. [3] compare CNN-AEs against a variety of Recurrent Autoencoders (RNN-AE) and supervised models, demonstrating that RNN-AEs provide superior anomaly detection performance for both Binary Black Hole (BBH) and Binary Neutron Star (BNS) mergers. Crucially, their work formalizes the inherent tradeoff between the strict accuracy of supervised methods, and the broad generalization capabilities of unsupervised strategies, laying down the base framework that our work seeks to extend.

III. DETECTORS AND SIGNAL

We use data simulated on the model of the LIGO detectors. They are designed to measure the strain caused by passing GWs. Using laser interferometry, these detectors measure relative changes in the lengths of two orthogonal arms extending for a couple kilometers, with a precision of the order of 10^{-21} m.

We perform this study on a sample of synthetic data, generated for [3], and publicly available on Zenodo [8], [9]. ‘Noise’ events occur when no signal is superimposed to the detector noise. Detector noise is generated with a power spectral density function tailored to the specification of the LIGO detectors, which is computed as a function of the length of the noise, and the time step of the noise using PyCBC. This approach to simulated data generation ignores all glitches and other transient sources of detector noise. Two kinds of GW sources from binary mergers are considered: binary black holes (BBH) and binary neutron stars (BNS). For the first variety, the coalescing masses are independently and uniformly sampled from $10 M_{\odot}$ to $80 M_{\odot}$, while for the second variety they are sampled from $1.1 M_{\odot}$ to $2.1 M_{\odot}$.

‘Signal’ events containing GWs are created by overlaying simulated GWs on top of the detector noise. This provides a scenario analogous to a true GW detection, in which the strain from the incoming wave is recorded in combination with the normal detector noise.

A. Datasets

The dataset is divided into two parts: one tailored to the H1 detector (Hanford), and the other to the L1 detector (Livingston). Both are composed of 325 000 noise samples and 25 000 signal injections. Each sample corresponds to 8 s of data, sampled at 2048 Hz, so a sample is represented as a one-dimensional array with 16 384 entries.

Band-pass filtering is applied to remove high frequencies (>2048 Hz) and low frequencies (<30 Hz) from the data. Background noise from outside the current interferometer sensitivity range is thus discarded. To facilitate the networks’ data processing and learning, the data are min-max scaled from 0 to 1.

Since each dataset is ~ 45 GB, and the available hardware is not powerful enough to efficiently process this amount of data, to improve training speed we select a subset from the L1 detector. This is made of 70 000 samples of noise for the training set, 12 000 samples of noise for the validation set, and 6000 samples of noise along with 6000 samples of signal injections for the test set. We are then left with 12 GB of data.

IV. PROCESSING PIPELINE

We train the networks on hardware provided by the Cloud-Veneto cloud infrastructure. We use a GPU NVIDIA Tesla T4, having 2560 CUDA cores divided in 40 streaming multiprocessors, 320 Tensor cores, and 16 GB of VRAM.

A. Models

For our analysis, we implement several distinct architectures. Each model undergoes hyperparameters’ optimization using the Optuna framework [10]. We then train the most promising configurations until convergence, and from this final subset, we select the networks presented here.

We start by replicating the best performing architecture from [3], from now on denoted as ‘UniAE’. We then experiment with several modifications to this architecture so as to try and achieve a better reconstruction of the input data.

We represent all of the networks’ schemes in Fig. 1 on page 4.

We use the largest possible batch size for each model according to the available VRAM.

We choose the window size for the univariate models to be 100—following the original implementation—with a stride of 60. In the case of the multivariate model, we set these as hyperparameters to tune, finally yielding a 48 window length, and 42 of stride.

1) *UniAE*: this model is an LSTM-based autoencoder designed to process univariate time series data. We build the encoder with two consecutive 3-layer LSTM modules: the first projects the input into a 32-dimensional hidden space, and the second compresses it into an 8-dimensional space. To form the bottleneck, the network extracts only the final hidden state of the encoder, producing an 8-dimensional vector that serves as the latent representation of the entire sequence.

Before the decoding phase, we repeat the latent representation 100 times to restore the original temporal dimension. The decoder processes this sequence through two 3-layer LSTM modules, where the first keeps the 8-dimensional space, and the second expands the hidden dimensions back to 32. Finally, we apply a 1D convolution with a kernel size of 1 as a step-wise linear projection, mapping the 32 features back to a single dimension to output the reconstructed univariate sequence. We train the network using batches of 64 sequences.

2) *DeepUniAE*: the first obvious modification is extending the network. We add a two-layer block of 128 hidden features to the beginning of the encoder, and mirror it at the end of the decoder. To avoid excessive computation time, we also reduce the 32-dimensional block to two layers. This model also includes dropout. We train it with a batch size of 16 samples because of the larger amount of weights.

3) *ResAE*: a more daring approach is the addition of skip connections between the encoder and the decoder. To avoid excessive information flowing outside of the bottleneck—hence undermining the whole training process—we make a single connection between the 8-dimensional blocks, the closest to the bottleneck. We train this network with a batch size of 64, like the original model.

4) *TDAE*: the last architecture we evaluate is a Time-Delay Autoencoder (TDAE) [11], which carries the heaviest changes from UniAE. This strategy transforms a univariate time series into a multivariate format [12] by concatenating signals across successive time samples to construct a new, multidimensional input sequence [11]. As demonstrated in [12], transforming univariate series into a multivariate representation via feature engineering can yield substantial performance gains without significantly increasing computational complexity. The network processes these concatenated time-delay sequences, which consist of 48 features over 389 windows, extracted using a stride of 42. The encoder module comprises four LSTM layers: the first two act as expansion layers, mapping the input features into a 768-dimensional space, while the subsequent two layers compress this representation down to 192 dimensions to extract the latent features. To stabilize the hidden states, we apply layer normalization immediately following each recurrent operation. We prefer Layer Normalization [13] for LSTMs over Batch Normalization because it normalizes across the feature dimension for each sample independently, namely it makes it time independent. Similar to the previously described networks, we aggregate the temporal information into a single bottleneck vector. To initiate the decoding process, we repeat this 192-dimensional latent representation across the predefined sequence length of 389. We initialize the decoder with the encoder’s final hidden state, and process the repeated vector through four LSTM layers that mirror the encoder’s architecture. Finally, we use a 1D convolution with a kernel size of 1 to act as a step-wise linear projection, mapping the recurrent output back to the original input feature space to generate the final reconstruction.

B. Loss

Given the nature of the problem, we use the mean squared error to measure the difference between the original data and the reconstruction. ResAE and DeepUniAE show a ‘collapsing’ issue, namely the reconstruction collapses into a flat mean of the input samples, without the networks learning the actual noise. To break this behaviour, we implement a regularization term consisting of the reciprocal of the standard deviation of the output. We apply this method to both models, adding a sigmoid activation function to their output, so as to limit the output values between 0 and 1.

Furthermore, we test other standard regularization and vanishing gradients fixes, such as gradient clipping, learning rate scheduling, and weight decay.

V. RESULTS

A. Hyperparameters

In Tab. 1 we show the best hyperparameters for each model.

Notably, the best optimizer resulted to be Adam or one of its variations. The only exception arose with the model ResAE trained with the regularized loss, which used RMSprop with a momentum of 0.7.

TABLE 1: Best hyperparameters selected from Optuna optimization. ‘Optim.’ is the chosen optimizer, and ‘LR’ is the starting learning rate. UniAE and TDAE were trained with no dropout. DeepUniAE* and ResAE* refers to the model trained with the regularized loss as described in Sec. IV-B.

Model	Optim.	LR	Clip	Dropout
UniAE	Nadam	9.4×10^{-4}	5.0	—
DeepUniAE	Adam	1.1×10^{-3}	1.0	0.5
DeepUniAE*	Nadam	3.1×10^{-2}	0.3	0.5
ResAE	Nadam	1.9×10^{-3}	1.9	0.25
ResAE*	RMSprop	2.7×10^{-3}	4.9	0.25
TDAE	AdamW	5.6×10^{-4}	2.0	—

B. Losses and Reconstructions

In Fig. 2 on the next page we show the loss curves of the models, and in Fig. 3 on page 5 the reconstructions of a noise sample taken from the test set.

DeepUniAE and ResAE showed the same loss and reconstruction, sticking to reproduce the mean of the input signal. Losses and reconstructions for these models trained with the regularized loss are not shown. The former have a different scale with respect to the other models’ losses, and would not be informative, the latter showed poor performance. Regularized DeepUniAE gave too much importance to the regularization part, hence outputting a binary signal to maximize the standard deviation. On the other hand we regularized ResAE by multiplying the new term by a 0.3 factor, but the model was not affected by the change in loss, and still output the mean of the signal (final loss ~ 30.0135).

UniAE emerged as the best model, for both loss value, and reconstruction power, and it was chosen to carry on the analysis, using the weights of the 235-th epoch, where the loss appears to be flat.

C. Anomaly Detection

In order to perform anomaly detection, we measured the ‘window wise’ mean squared error between the test samples and UniAE’s reconstructions, computing a score for each window of every sample of the test set. The residuals for the second half of the test set (see below) are shown in Fig. 4 on page 5.

To choose a suitable threshold for the anomalies, we further divided the test set in half, evenly splitting noise and signal samples. We then used the first half to compute precision and recall scores for a range of possible thresholds. These scores are shown in Fig. 5.

A desirable tradeoff between the two is dependent on the specific employment of the network in a larger analysis pipeline. We chose 1.42×10^{-2} as the threshold, favoring recall (75 %) over precision (63 %), not to miss too many GWs.

Finally we tested the chosen threshold on the other untouched half of the test set, obtaining similar and adequate results, namely 76 % of recall, and 64 % of precision.

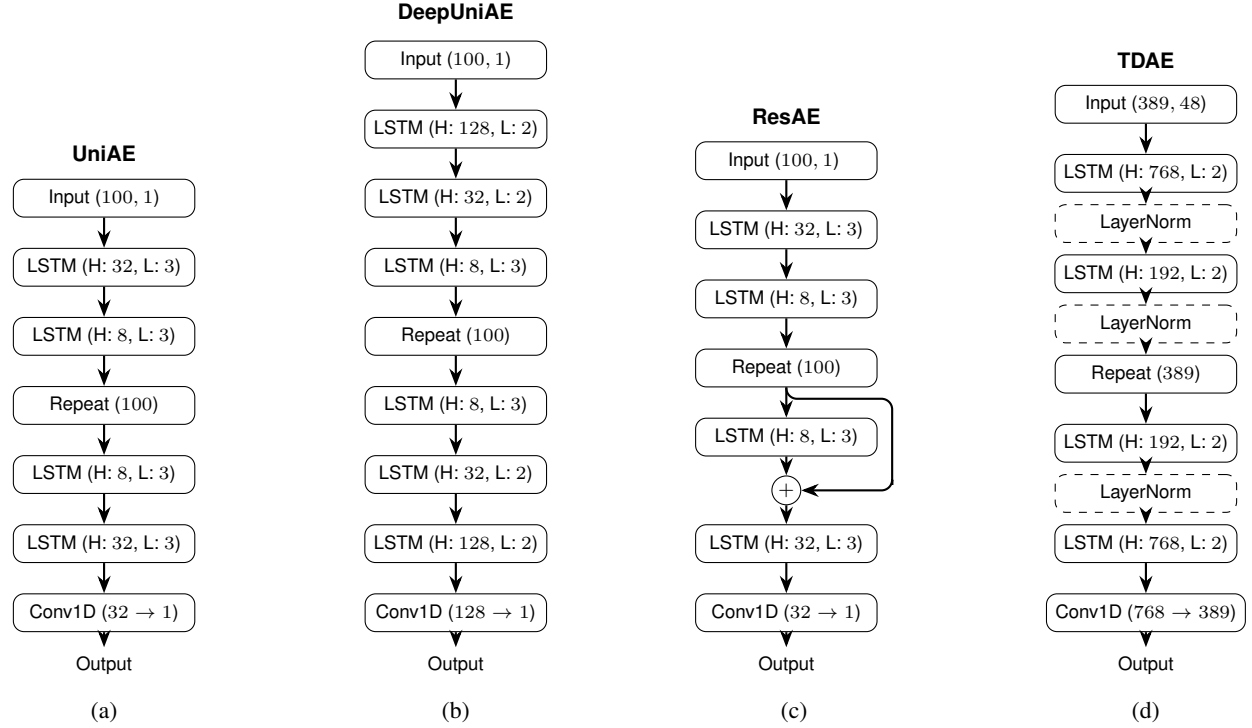


Fig. 1: The architectures we present in this review. ‘H’ is the number of features in the LSTM hidden states, and ‘L’ is the number of consecutive layers. From left to right: *UniAE* is built as the best performing network from [3], *DeepUniAE* is a deeper version of the former, *ResAE* tries to tackle some learning issues by implementing a skip connection, while *TDAE* tries to solve the same problem by switching from univariate TSs to multivariate TSs.

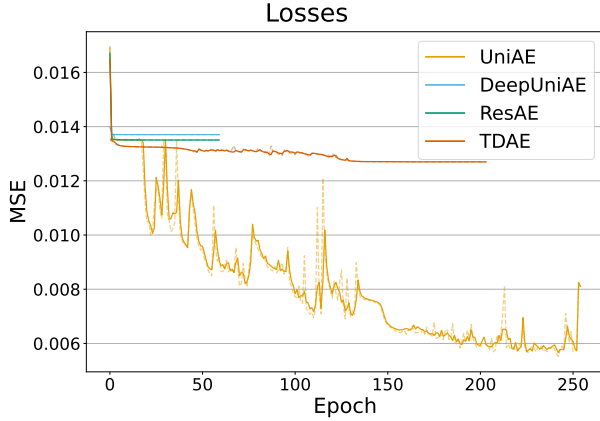


Fig. 2: Training and validation loss curves for each trained model. The former are the solid lines, the latter are the dashed lines. DeepUniAE curves were offset by $+2 \times 10^{-4}$ to avoid overlapping with ResAE curves.

VI. CONCLUDING REMARKS

In this study, we explored the feasibility of an LSTM-based autoencoder for anomaly detection in gravitational wave time series, building upon the work of Moreno et al. [3]. We reproduced their best performing architecture (UniAE, Sec. IV-A1 on page 2), and developed several variations aiming to enhance performance and reconstructions:

- DeepUniAE (Sec. IV-A2) with additional layers.
- ResAE (Sec. IV-A3) testing skip connections.
- TDAE (Sec. IV-A4) shifting from a univariate to a multivariate paradigm to leverage performance gains reported in the literature.

Ultimately, the baseline model from Moreno et al. [3] outperformed the modifications we tried, which suffered from lower reconstruction accuracy, and consequently reduced detection capability.

To address the flatline problem, we introduced an amplitude regularization term to the loss function, which did not eventually prove to enhance the reconstructions. We nonetheless believe it could be better tuned and exploited in the time to come.

For future work, we planned to further investigate incorporating such inductive biases through a fully physics-informed approach. Additionally, while this study relied on synthetic monofrequency data, we intended to utilize real, multifrequency datasets to fully exploit multivariate architectures. Different models could be explored as well such as Temporal Convolutional Networks [14], Variational Autoencoders, and Generative Adversarial Networks.

Replicating the methodology of Moreno et al. presented significant challenges, partly due to insufficient documentation regarding their synthetic dataset generation, and experimental procedures. We questioned several of their methodological

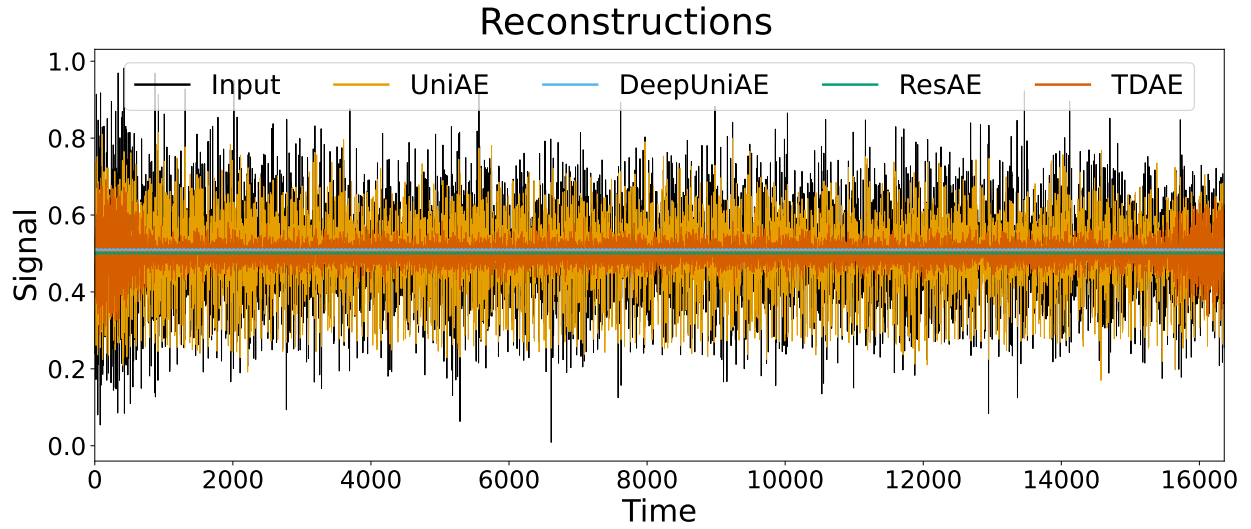


Fig. 3: Reconstruction of each model of a noise sample labeled ‘Input’ taken from the test set. DeepUniAE reconstruction was offset by +0.01 to avoid overlapping with ResAE’s one.

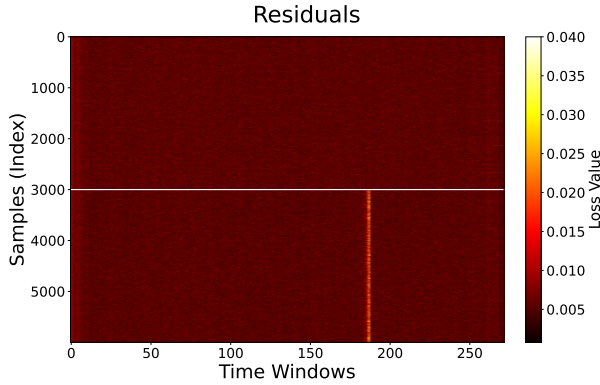


Fig. 4: Window wise mean squared error between the test set samples and UniAE’s reconstructions. The white horizontal line separates noise samples (above the line) and signal injection samples (below the line). A brighter color denotes a larger difference and a possible anomaly.

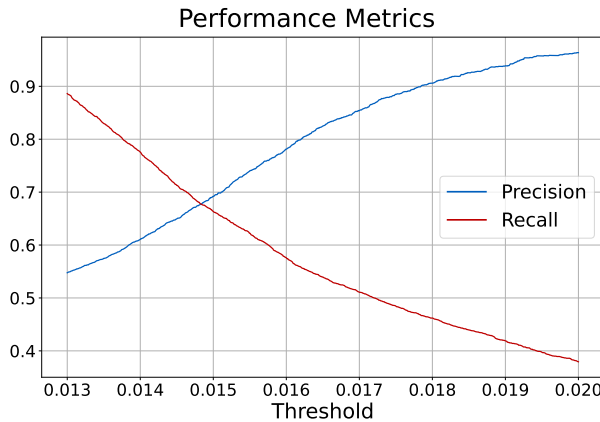


Fig. 5: Precision and recall curves as a function of candidate thresholds using UniAE.

choices. Our difficulty in reproducing their exact model performance could also be attributed to our use of a data subset, a necessary compromise due to the prohibitive training times required by the full dataset.

Furthermore, this project provided us with valuable expertise in anomaly detection, a highly versatile technique applicable across diverse domains, and it demonstrated how a broader portfolio of deep learning methodologies could be effectively applied to complex problem-solving.

REFERENCES

- [1] B. P. Abbott, R. Abbott *et al.*, “Prospects for observing and localizing gravitational-wave transients with advanced ligo, advanced virgo and kagra,” *Living Reviews in Relativity*, vol. 23, no. 1, Sep. 2020. [Online]. Available: <http://dx.doi.org/10.1007/s41114-020-00026-9>
- [2] LIGO Scientific Collaboration. Gravitational-wave candidate event database. [Online]. Available: <https://gracedb.ligo.org/superevents/public/O4>
- [3] E. A. Moreno, B. Borzyszkowski *et al.*, “Source-agnostic gravitational-wave detection with recurrent autoencoders,” 2021. [Online]. Available: <https://arxiv.org/abs/2107.12698>
- [4] C. Ma, S. Wang *et al.*, “Using deep learning to predict matched signal-to-noise ratio of gravitational waves,” *Physical Review D*, vol. 109, no. 4, 2024. [Online]. Available: <http://dx.doi.org/10.1103/PhysRevD.109.043009>
- [5] M. Zevin, S. Coughlin *et al.*, “Gravity spy: integrating advanced ligo detector characterization, machine learning, and citizen science,” *Classical and Quantum Gravity*, vol. 34, no. 6, p. 064003, Feb. 2017. [Online]. Available: <http://dx.doi.org/10.1088/1361-6382/aa5cea>
- [6] D. George, H. Shen, and E. A. Huerta, “Classification and unsupervised clustering of ligo data with deep transfer learning,” *Phys. Rev. D*, vol. 97, p. 101501, May 2018. [Online]. Available: <https://link.aps.org/doi/10.1103/PhysRevD.97.101501>
- [7] F. Morawski, M. Bejger *et al.*, “Anomaly detection in gravitational waves data using convolutional autoencoders,” *Machine Learning: Science and Technology*, vol. 2, no. 4, p. 045014, jul 2021. [Online]. Available: <https://doi.org/10.1088/2632-2153/abf3d0>
- [8] E. Moreno, “Source-agnostic gravitational-wave detection with recurrent autoencoders: L1 detector,” Jul. 2021. [Online]. Available: <https://doi.org/10.5281/zenodo.5121510>
- [9] E. Moreno, “Source-agnostic gravitational-wave detection with recurrent autoencoders: H1 detector,” Jul. 2021. [Online]. Available: <https://doi.org/10.5281/zenodo.5121514>

- [10] T. Akiba, S. Sano *et al.*, “Optuna: A next-generation hyperparameter optimization framework,” in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, ser. KDD '19. New York, NY, USA: Association for Computing Machinery, 2019, pp. 2623–2631. [Online]. Available: <https://doi.org/10.1145/3292500.3330701>
- [11] B. Liu, J. Guo *et al.*, “Anomaly detection and diagnosis using pre-processing and time-delay autoencoder,” in *2021 26th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)*, 2021, pp. 1–8.
- [12] J. Audibert, S. Marti *et al.*, “From univariate to multivariate time series anomaly detection with non-local information,” in *Advanced Analytics and Learning on Temporal Data*, V. Lemaire, S. Malinowski *et al.*, Eds. Springer International Publishing, 2021, pp. 186–194.
- [13] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer normalization,” 2016. [Online]. Available: <https://arxiv.org/abs/1607.06450>
- [14] M. Thill, W. Konen *et al.*, “Temporal convolutional autoencoder for unsupervised anomaly detection in time series,” *Applied Soft Computing*, vol. 112, p. 107751, 2021. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1568494621006724>